PREPARATION / NOT PART OF THE RECORDING

A 19-minute Lea demonstration

Leave one minute for compilation, network, or model latency.

This script demonstrates the Lea extension inside Overleaf. Quoted paragraphs are suggested narration; numbered steps are actions. Controls depend on the item's current state. A reference proof or a finished example is not evidence that an agent just completed a live run.

Files to keep open

- Upload `lea-overleaf-project.zip` as an Overleaf project. Keep `main.tex` as its main document and work in Code mode.
- Keep `copy-paste-snippets.txt` beside this script for the exact edits.
- Keep a second, independently rehearsed Overleaf project ready as a backup. A copied Overleaf project has a different Lea identity; prepare its Lea results separately rather than assuming history transfers.

Prepare the states before recording

1. Start the normal Lea services, load the extension, compile the document, open the floating **L**, and refresh the pane. Expect **9 marked items**. The unmarked Doubling zero lemma and the unmarked Shift identity are intentional.
2. Formalize `demo_double`, then `demo_double_add`, then `demo_double_two`. Inspect the actual names, namespaces, types, and imports. Rehearse snippets F–H so the corollary genuinely depends on the lemma. Restore explicit arguments and a checked corollary for the recording.
3. Leave Doubling zero unmarked for the live insertion. Leave the two optional input examples unfinished for the batch demonstration.
4. Enable **Pause formalization when Lea needs author input** before individually formalizing the flawed square claim. Preserve its real finding or checked disproof. If paused, confirm there is a usable saved checkpoint.
5. Formalize `demo_shifted_square`, then `demo_shift_one`, then `demo_shift_two`. Require an actual shared helper, its checked status, and both consumers' imports before describing the example as reuse.
6. Keep a past batch report available. Rehearse ZIP export. Configure a demonstration GitHub remote beforehand only if you intend to show a real push or import. Keep saved credentials concealed throughout the recording.

Time checkpoints

0–3 min: open and mark. 3–6: formalize and review. 6–8: chat and edit. 8–11: fix source. 11–13: repair. 13–15: batch. 15–17: shared results and Blueprint. 17–19: settings, export, and close.

Latency rule: spend at most about 20 seconds waiting without something useful to explain. Then say, "That run is still working. Here is the same workflow on a result I prepared earlier." Switch only to a genuinely prepared state.

0:00–3:00

Open Lea and mark a statement

Start on page 1 of the compiled document, with main.tex open.

Click and do

1. Click the floating **L** button (**Lean project pane**). Resize the pane so both source and item details remain readable.
2. Expand `main.tex` and its section. Point out the other two source files and their progress indicators. Use **Go to source** on Doubling a sum.
3. In `main.tex`, find `DEMO LIVE INSERT` inside the Doubling zero lemma. Paste snippet A as one new line:

```
% lea: formalize label=demo_double_zero uses={demo_double}
```

4. Let Overleaf save. Refresh the Lea pane if the item has not appeared. Expect **10 marked items**. Click the source badge and show **Show in Lean pane**, then return to the item's details.
5. Point to the definition's `\leadefinition`, the custom `claim` environment's `\leatheorem`, and the corollary's `uses` and `context` fields. The package is already included in this project; explain **Add Lea tags** if that setup control is offered.

Say

"This is a small mathematical document about doubling a natural number. Lea adds a formalization workspace alongside the source. I choose which mathematical blocks it should work on, and I can jump between the document and the corresponding formal result."

"This lemma was ordinary LaTeX. Adding one comment asks Lea to formalize it; the comment does not change the PDF. Inline tags are another supported form. They also let me use my own environment names, such as this Claim."

"The dependency field points to earlier Lea items. The context field gives guidance, such as notation or a proof strategy. Explicit labels make this demo easy to follow; they are optional when I just want to mark a new item."

If the state differs

If the pane is empty, refresh after the source saves and check that the extension is connected. Do not insert another marker in an already tagged block. If the live marker is already present from rehearsal, remove that one line before recording.

3:00–6:00

Formalize and inspect the evidence

Live target: demo_double_zero. Prepared results: definition and additivity.

Click and do

1. Expand Doubling zero. Under **Other actions**, open the more-actions menu and choose **Stub** if offered. Show the skeleton and its unfinished obligation. This step is optional; **Formalize** works directly.
2. Click **Formalize** (or **Re-formalize** after creating the stub). Expand the live progress/status view. Point out **Pause**; use it only if the run is still active and the pause/resume path was rehearsed.
3. While it runs, expand the prepared Doubling definition and show that it produces a Lean definition with a body, not a theorem requiring a proof.
4. Return to Doubling zero when it settles, or explicitly switch to the prepared Doubling a sum result. Show **Lean code** and its copy control.
5. Inspect **Lean Check**. Expand **Lea Status details**, read the assessment's scope and any findings, then open **Update history**.
6. Compare the Lean proposition with the LaTeX. On an eligible checked result you have actually reviewed, click the personal **checkmark**.

Say

"A stub gives me a formal statement to work from, but it is still unfinished. Formalize asks Lea to complete the task. I can follow its progress, and pause an individual run when necessary."

"Here is the resulting Lean. I check the types, hypotheses, and conclusion against the mathematical statement. In this example the domain is the natural numbers and the conclusion says that doubling zero gives zero."

"These two indicators answer different questions. Lean Check reports the formal checking result. Lea Status reports Lea's assessment of the source and formalization, including findings or remaining obligations. I read the scope: an assessment of partial work is not a completed proof."

"This checkmark records my personal review of this revision and its dependencies. It is separate from formal checking, and later changes can invalidate it."

If the state differs

If a run finishes before Pause can be used, leave the completed result intact. Show a genuine prepared paused item later. Do not call an unchecked artifact verified, or describe confidence alone as a correctness guarantee.

6:00–8:00

Ask a question, then edit the proof

Prepared target: demo_double_add with explicit arguments m and n.

Click and do

1. Expand Doubling a sum. In **Other actions**, click **Chat**. Paste snippet B and send it:

```
Explain how the Lean statement matches the LaTeX claim and how the proof
rearranges addition. Do not change any files.
```

2. Read a short portion of the response if it arrives. Point out **Open in LeaChat**, which continues the same session in the larger UI. Keep the recording focused on the extension.
3. Wait until the item's chat run is idle before editing. Choose **Edit** from the more-actions menu. Preserve the generated imports, namespace, declaration name, and statement. Replace only the proof after `:=` with snippet C:

```
by
  simp only [demo_double]
  ring
```

4. Click **Save**. Inspect the checking result and dependent impact summary. If the generated definition has a qualified name, use the exact name rehearsed for this project. Point out **Cancel** for discarding a draft.

Say

"I can ask questions in the context of this item. Here I want an explanation of how the formal statement and proof correspond to the source. The session also remains available in LeaChat."

"I can edit the Lean myself. This replacement keeps the same mathematical statement: it unfolds doubling and lets the ring tactic rearrange the arithmetic. Saving records and checks my edit, including affected dependents when needed. It does not start another agent proof search."

If the state differs

If chat is slow, show the prepared conversation and identify it as earlier work. Do not edit during its active run. If the generated names or statement differ from the rehearsed version, use a prepared safe edit or show Edit/Cancel instead of pasting a mismatched declaration.

8:00–11:00

Find a source problem and correct it

Target: demo_square_bound in sections/02-source-review.tex.

Click and do

1. Use **Go to source** on Square versus the original number. Show the original claim and its associated proof, marked as a flawed exercise.
2. Open the genuine finding prepared before recording. If demonstrating a new run, first enable **Pause formalization when Lea needs author input** in the gear menu, save, and start this item individually.
3. Expand **Lea Status details**. Explain the failed sign argument. If the outcome is **Claim disproved**, inspect whether the counterexample itself is checked. If paused, show the actual **Resume** control.
4. In the source editor, replace the whole theorem and following proof with **snippet D**. This adds $x \geq 1$ and corrects the sign argument. Keep the Lea label `demo_square_bound` unchanged. Let Overleaf save.
5. For a compatible paused run, click **Resume**. For a completed disproof or another settled run, choose **Re-formalize**. These are different actions; describe the one actually used.
6. Inspect the new Lean statement: it must contain the new hypothesis $x \geq 1$. If a genuine review candidate is offered, inspect it before **Approve result**. That applies the reviewed candidate; it is separate from the personal review checkmark.

Say

"The original statement says that every real number is at most its square. That is false: at one half, the square is one quarter. The author's proof also incorrectly assumes that both factors are always nonnegative."

"Lea should surface the issue rather than quietly insert the missing hypothesis. Depending on the run, that can appear as a request for author input or as a checked counterexample to the original claim."

"I am now changing the mathematics explicitly. With the assumption that x is at least one, both factors are nonnegative and the proof works. I check that the new formal statement includes that assumption."

Optional, if time remains After the corrected result is checked, use snippet E to rewrite the displayed conclusion as $x^2 - x \geq 0$. Observe outdated status, then **Re-formalize**. Show that an earlier personal review or assessment does not stay current merely because a proof was once checked.

If the state differs

If Resume reports an incompatible checkpoint, keep the message visible and use Re-formalize as a fresh attempt. This example has an author proof, so it is not the example for the conditional **Continue best effort** action.

11:00–13:00

Inspect a broken dependent and repair it

Use the rehearsed additivity-to-corollary dependency.

Click and do

1. Confirm that Doubling a sum and Adding two before doubling are checked and idle. Open the corollary's Lean code. It must really apply `demo_double_add n 2`; snippet F prepares that proof if needed.
2. Edit `demo_double_add`. Change only its binder notation from `(m n : Nat)` to `{m n : Nat}` as in snippet G. Preserve its proposition, name, proof body, imports, and namespace. Click **Save**.
3. Inspect the impact summary and the dependent corollary's error. The lemma remains mathematically the same, but its old explicit application is incompatible with the new implicit arguments.
4. Click **Repair with Lea** on the affected corollary. Point out **Repair all** if a group is offered. Inspect the resulting code and current Lean Check when the repair settles.

Say

"These results are connected by actual Lean dependencies. I am making a small interface change: the lemma's two arguments become implicit. Its mathematical content stays the same, but an existing caller may need updating."

"Lea reports the affected work instead of continuing to present its old check as current. I can ask Lea to repair the dependent result and then inspect what changed."

Deterministic manual fallback If agent repair is slow, use a previously completed repair. Alternatively, when no agent run is active, edit the dependent lemma application as in snippet H:

```
-- Before:
demo_double_add n 2
-- After:
demo_double_add (m := n) (n := 2)
```

Click **Save** and inspect the check. Say, "Here I repaired the call manually; Repair with Lea automates that kind of adaptation."

If the state differs

This is an optional live breakage demonstration, not a required production workflow. If the generated corollary does not call the lemma, do not claim the edit broke it. If the save is rejected or the calling convention was not rehearsed, show the prepared impact and repair example. Avoid leaving a live repair active when starting the next batch.

13:00–15:00

Run a batch and inspect its report

Small remaining targets: standalone proposition and existing Lean.

Click and do

1. Open the optional input forms near the bottom of `main.tex`. Point out the standalone proposition tag and the `leacode` block. These are already marked, but can be left unfinished until this step.
2. Return to the top of **Items**. Point out **Stub all** if eligible theorems remain. Click **Formalize all** when the project is idle and the source corrections and repairs have settled.
3. Show the current item, queued work, and outcomes. Explain that eligible items are ordered by declared dependencies and completed work is not simply restarted. If shared consumers are still unfinished, expect their supporting work to appear as well.
4. Point out **Stop** while the batch is active. Demonstrate it only if that stop/continuation was rehearsed. Use **Continue remaining** only when the actual paused batch offers it.
5. When the batch settles, open **Formalize all report**. If it is still running, open **Past Formalize all reports** and identify the selected report as an earlier batch. Inspect outcomes and shared-result counts.

Say

"For a larger document I can formalize eligible items as a batch, following their declared dependencies. The panel shows what is running, what remains, and what needs attention."

"A finished batch is not a promise that every item succeeded. I read the report for checked results, failures, findings, and shared mathematics. Source findings stay visible during Formalize all; the batch does not use the individual-run pause-on-author-input behavior."

"These two extra inputs show that I can mark standalone prose or supply existing Lean code. Code appearing in the PDF is only source material; I still inspect Lea's checking result."

If the state differs

If no eligible items remain, show the saved report rather than deliberately invalidating a checked result. Do not claim the initial marked-item count is a count of newly completed proofs. A stub with an unfinished obligation remains unfinished even if a stub-generation batch succeeds.

15:00–17:00

Reuse mathematics and view the Blueprint

Prepared shared example: shifted squares.

Click and do

1. Open `sections/03-shared-results.tex`. Show the definition $Q_a(x) = (x-a)^2$, the unmarked Shift identity $Q_a(a+t) = t^2$, and its two consumers at $t = 1$ and $t = 2$.
2. Expand **Shared mathematics**. Select the helper produced or reused by the rehearsed run. Inspect its full signature, checked state, canonical module/import, attempts, and **Used by** consumers.
3. Open both consumer artifacts. Confirm they import and use that same public helper. If a genuine dependency wait is present, use **View supporting result** and explain automatic continuation. Point out **Refresh status**, **Stop waiting**, or **Continue when ready** only when offered in that state.
4. Switch from **Items** to **Blueprint**. Click **Generate from formalized theorems** if a starter graph is needed. Select a doubling node and inspect its kind, status, and dependencies.

Say

"Both of these results use the same supporting identity. The identity is source mathematics here; it is not an extra tagged document target. Lea can create a shared formal result and reuse it for multiple consumers."

"I verify reuse in the shared record and the generated imports. If the helper is already checked, immediate reuse is expected. If it has to be proved, the requesting item can wait and continue when the supporting work is ready."

"Blueprint gives me a project view of the mathematical dependencies. I can move from individual proofs to the structure of the formalization and inspect which pieces are checked or still need work."

If the state differs

The source guidance requests shared-helper use, but agent choices are not a deterministic test harness. If the live run proves each consumer independently, say so; it has not demonstrated shared reuse. Use the prepared genuine shared record. Do not invent a new helper attempt when an existing one was reused.

17:00–19:00

Show controls, export, and finish

Keep this final tour short. Reserve 19:00–20:00 for spillover.

Click and do

1. Click the gear for **Lea settings and usage**. Show usage and maximum spend, then model selection, maximum turns, source mirroring, and the individual-run author-input pause option. Keep configured API keys hidden.
2. Show the project name and Lean namespace. Open **Edit project name** or **Edit name**, inspect the preview, then cancel. Avoid renaming the rehearsed project during this recording.
3. Open **Share**, then **Download .zip**. Confirm the Lean project export is offered or downloaded. Explain that it contains the Lean project, distinct from the LaTeX project used as source.
4. Show the configured remote and **Push to GitHub**. Execute a push only to the intended demo repository if it was prepared for the recording. Otherwise identify this as a tour of the control.
5. Show **Add Lean files from GitHub**. With a prepared repository, inspect an actual import preview and its skipped conflicts before deciding whether to confirm. Imported work still needs the recorded checking process.
6. Close the panel and return to page 1 with a checked result visible.

Say

"I can choose the model, review usage, and control the run and spend limits from the extension. Source mirroring makes surrounding LaTeX available as context. The project also has a name and Lean namespace."

"The formal work is portable: I can download the Lean project, push it to a configured GitHub repository, or preview Lean files to add from GitHub. For an import I inspect the preview and subsequent checks."

"We started with mathematical prose, selected the statements to formalize, and inspected the Lean and the source findings. We then refined a proof, corrected a claim, handled dependencies, and exported the resulting project."

If the state differs

Do not spend recording time configuring credentials or hunting for a repository. If there is no prepared remote, show the available actions and explain them without claiming a push or import occurred. The ZIP export is the concrete portable result to finish on.

REHEARSAL NOTES

Cuts, feature coverage, and evidence

Use this page before recording, not as a slide.

For a 15-minute version Keep the same sequence. Use these checkpoints:

- 0–2: open and mark; 2–5: live formalization and review.
- 5–6: chat or a safe manual edit; 6–9: the source correction.
- 9–10: show the prepared repair outcome; 10–12: batch and its report.
- 12–14: shared mathematics and Blueprint; 14–15: ZIP export and close.

Cut live dependency breakage, the optional second source revision, and the GitHub/configuration tour first. Keep the check/status distinction and the author's review of the mathematical statement.

What is covered

- Source navigation, file grouping/progress, inline badges, comments, definition/theorem/corollary tags, a custom environment, standalone prose, and an existing-Lean block.
- Labels, declared dependencies, guidance, associated author proofs, optional stubs, individual formalization, and pause/resume controls.
- Lean Check, live Lea Status and history, outdated source, a counterexample or source finding, source corrections and new formalizations, conditional result review, and personal review tied to a revision.
- Item chat, the LeaChat session link, manual edits/checks, dependent impact, individual/group repair, batch actions, saved reports, shared helper reuse, conditional dependency-wait controls, and Blueprint.
- Model/usage/limits, source context, project identity, ZIP export, and the GitHub push/import interfaces.

Conditional features Not every state is expected in one recording. **Approve result** needs a real review candidate. **Continue best effort** needs an eligible source pause without an associated author proof; the flawed-claim example deliberately has a proof. **Continue remaining** needs a paused batch. Waiting controls need a real shared dependency wait. Mention these only in their proper context or use separately rehearsed examples.

What was validated when preparing this kit The LaTeX projects are compiled, the marker inventory and proof associations are checked against the extension parser, and the reference Lean proofs are checked locally using the repository's Lean/Mathlib toolchain. The reference file also checks the false claim's counterexample and the repaired implicit argument call. These checks do not certify an unrun agent-generated artifact, guarantee model timing, or replace the live Overleaf rehearsal above.